Multi-Agent Systems

Orchestrator-Worker · Parallel Execution · Agent Communication · Specialized Workers · Production Deployment · Evaluation

WEEK 3 · DAY 20 **Saturday, July 25 2026** **Day 20 of 84**

Sec	Topic	Coverage
Sec 1	Orchestrator-Worker Architecture	AgentRole, AgentMessage, TaskSpec, AgentResult
Sec 2	Specialized Worker Agents	RetrievalAgent, GenerationAgent, ValidationAgent
Sec 3	Orchestrator	4-phase pipeline: parallel P1 + gen + validate + execute
Sec 4	Parallel Execution Patterns	gather, fan-out, sequential+exit, speculative
Sec 5	Agent Communication Protocol	AgentBus, MSG_TYPES, trace_id, pub/sub
Sec 6	Multi-Agent Evaluation	MultiAgentMetrics, SLOs, per-phase latency targets
Sec 7	Production Deployment	AgentPool, semaphore, FastAPI endpoint, health check
Sec 8	FAANG Patterns	Google/Microsoft/Amazon/Meta/Netflix multi-agent patterns
Sec 9	Architect's Lens	When to use, model sizing, failure isolation, stateless
Sec 10	Exercises + Summary	Timeline tracing, failure injection, model comparison

TEXT-TO-SQL AGENT COMPLETE (Days 13-20): D13 foundation + D14 vLLM/FastAPI + D15 RAG schema + D16 indexing + D17 hybrid retrieval + D18 generation + D19 single agent loop + D20 multi-agent orchestration (parallel phases, specialized workers, agent pools, production deployment). Multi-agent SLOs: success>95%, latency<4s, retries<15%.

Section 1 — Orchestrator-Worker Architecture

A multi-agent system routes work to specialized agents coordinated by an orchestrator. Agents communicate via typed messages. Each agent has a single role.

```
from enum import Enum
from dataclasses import dataclass, field
import time

class AgentRole(Enum):
    ORCHESTRATOR = "orchestrator"
    RETRIEVAL    = "retrieval"
    GENERATION   = "generation"
    VALIDATION   = "validation"
    CRITIC       = "critic"

@dataclass
class AgentMessage:
    """Unit of communication between agents."""
    sender:    AgentRole
    recipient: AgentRole
    msg_type:  str         # "task", "result", "error", "clarify", "progress", "cancel"
    payload:   dict
    trace_id:  str         # shared across ALL agents for one user request
    timestamp: float = field(default_factory=time.time)

@dataclass
class TaskSpec:
    """What the orchestrator hands to a worker agent."""
    trace_id:    str
    role:        AgentRole
    question:    str
    schema_name: str
    tenant_id:   str
    context:     dict = field(default_factory=dict)
    # context carries outputs from agents that completed before this one

@dataclass
class AgentResult:
    """What a worker agent returns to the orchestrator."""
    trace_id:   str
    role:       AgentRole
    success:    bool
    output:     dict
    latency_ms: float
    error:      str | None = None
```

Section 2 — Specialized Worker Agents

Each worker does exactly one job. Narrow scope = better prompt = better results. Workers can use different model sizes (small for retrieval/validation, large for generation).

```
class RetrievalAgent(BaseWorkerAgent):
    """SOLE JOB: retrieve + filter schema to exactly what's needed."""
    SYSTEM = (
```

```
        "You are a schema retrieval specialist. Given a question and raw DDL, "
        "identify: needed_tables, relevant_columns, implied_joins, schema_context. "
        "Return JSON: {needed_tables, relevant_columns, implied_joins, schema_context}")

    async def run(self, task: TaskSpec) -> AgentResult:
        # 1. Raw retrieval from HybridSchemaRetriever (Day 17)
        tables = await self.retriever.retrieve(task.question, task.schema_name,
                                               task.tenant_id, top_n=8)
        # 2. LLM filters to exactly needed tables+columns
        raw = await self._call_llm([
            {"role":"system","content":self.SYSTEM},
            {"role":"user",  "content":f"QUESTION:{task.question}\nSCHEMA:{schema_text}"}])
        # Returns: {schema_context, needed_tables, relevant_columns, implied_joins}

class GenerationAgent(BaseWorkerAgent):
    """SOLE JOB: generate SQL with CoT from pre-filtered schema."""
    SYSTEM = (
        "You are a SQL generation specialist. Use 5-step chain-of-thought: "
        "Step1=tables, Step2=columns, Step3=joins, Step4=filters, Step5=SQL. "
        "Return JSON: {reasoning, sql, tables_used, columns_used}")

    async def run(self, task: TaskSpec) -> AgentResult:
        schema_context = task.context.get("schema_context", "")
        few_shot       = task.context.get("few_shot_examples", "")
        # Receives filtered schema from RetrievalAgent — no raw retrieval here
        raw = await self._call_llm([
            {"role":"system","content":self.SYSTEM},
            {"role":"user",  "content":f"SCHEMA:\n{schema_context}\n"
                                      f"EXAMPLES:\n{few_shot}\n"
                                      f"QUESTION:{task.question}"}], max_tokens=600)
        # Returns: {reasoning, sql, tables_used, columns_used}

class ValidationAgent(BaseWorkerAgent):
    """SOLE JOB: validate SQL via EXPLAIN + LLM semantic check."""
    async def _explain(self, sql: str) -> dict:
        """Run EXPLAIN against real DB — catches syntax errors."""
        conn  = await asyncpg.connect(self.db_url)
        rows  = await conn.fetch(f"EXPLAIN {sql}")
        await conn.close()
        return {"valid":True, "plan":"\n".join(r[0] for r in rows)}

    async def run(self, task: TaskSpec) -> AgentResult:
        sql = task.context.get("sql","")
        # Pass 1: EXPLAIN (real DB syntax check)
        explain = await self._explain(sql)
        # Pass 2: LLM semantic validation (hallucinated columns/tables)
        raw = await self._call_llm([...])
        # Returns: {is_valid, issues, corrected_sql, explanation}
```

Section 3 — TextToSQLOrchestrator

Execution graph: Phase 1 (retrieval + few-shot IN PARALLEL) → Phase 2 (generation) → Phase 3 (validation, up to 2 retries) → Phase 4 (execute). Phase 1 is parallel — the key latency win.

```
class TextToSQLOrchestrator:
    MAX_VALIDATION_RETRIES = 2

    async def run(self, question, schema_name, tenant_id, user_id) -> dict:
        trace_id = str(uuid.uuid4())[:8]
        t_start  = time.perf_counter()
        timeline = []

        # ■■ Phase 1: PARALLEL retrieval + few-shot fetch ■■■■■■■■■■■■■■■■■■■■
        retrieval_result, past_sqls = await asyncio.gather(
            self.retrieval.run(TaskSpec(trace_id=trace_id, role=AgentRole.RETRIEVAL,
                question=question, schema_name=schema_name, tenant_id=tenant_id)),
            self.lt_memory.find_similar_past_sql(tenant_id, question, top_k=3),
        )
        # Both run simultaneously: ~250ms total vs ~450ms sequential (44% faster)
        if not retrieval_result.success:
            return self._error(trace_id, "Retrieval failed", retrieval_result.error, timeline)

        schema_context = retrieval_result.output["schema_context"]
        few_shot_text  = "\n".join(f"Q: {p['question']}\nSQL: {p['sql']}" for p in past_sqls)

        # ■■ Phase 2: Generation (needs Phase 1 output) ■■■■■■■■■■■■■■■■■■■■■■■
        gen_result = await self.generation.run(TaskSpec(
            trace_id=trace_id, role=AgentRole.GENERATION,
            question=question, schema_name=schema_name, tenant_id=tenant_id,
            context={"schema_context":schema_context, "few_shot_examples":few_shot_text}))
        if not gen_result.success:
            return self._error(trace_id, "Generation failed", gen_result.error, timeline)
        sql = gen_result.output.get("sql","")

        # ■■ Phase 3: Validation (with retry on failure) ■■■■■■■■■■■■■■■■■■■■■■■
        for attempt in range(self.MAX_VALIDATION_RETRIES + 1):
            val_result = await self.validation.run(TaskSpec(
                trace_id=trace_id, role=AgentRole.VALIDATION,
                question=question, schema_name=schema_name, tenant_id=tenant_id,
                context={"sql":sql, "schema_context":schema_context}))
            if val_result.output.get("is_valid", True): break
            sql = val_result.output.get("corrected_sql", sql)  # use corrected SQL

        # ■■ Phase 4: Execute ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
        exec_result = await self.toolkit.execute_sql(sql=sql, tenant_id=tenant_id)
        if "error" in exec_result:
            return self._error(trace_id, "Execution failed", exec_result["error"], timeline)

        # Persist successful SQL to long-term memory
        await self.lt_memory.save_successful_sql(tenant_id, user_id, question, sql)
        return {"trace_id":trace_id,"sql":sql,"rows":exec_result["rows"],
                "total_latency_ms":(time.perf_counter()-t_start)*1000,"success":True}
```

Section 4 — Parallel Execution Patterns

```
# ■■ Pattern A: Parallel independent agents (asyncio.gather) ■■■■■■■■■■■■■■■
# Use when: agents share the same input, produce independent outputs
retrieval_result, past_sqls = await asyncio.gather(
    retrieval_agent.run(task),
    lt_memory.find_similar_past_sql(tenant_id, question),
)

# ■■ Pattern B: Fan-out + merge ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
# Use when: multiple sources searched simultaneously
dense_r, sparse_r, docs_r = await asyncio.gather(
    dense_retriever.retrieve(question, schema_name, tenant_id),
    sparse_retriever.retrieve(question, schema_name, tenant_id),
    docs_retriever.retrieve(question, tenant_id),
)
merged = rrf_merge([dense_r, sparse_r, docs_r])  # RRF merge from Day 17

# ■■ Pattern C: Sequential with early exit ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
# Use when: later agents need earlier agents' output; failure short-circuits
async def sequential_with_exit(agents, task):
    context = {}
    for agent in agents:
        task.context.update(context)
        result = await agent.run(task)
        if not result.success:
            return None, result.error   # stop immediately
        context.update(result.output)
    return context, None

# ■■ Pattern D: Speculative execution ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
# Use when: multiple strategies compete; fastest valid result wins
# Amazon Bedrock pattern: 5 candidate SQL generations → best wins
async def speculative_generate(agents, task):
    tasks = [asyncio.create_task(agent.run(task)) for agent in agents]
    done, pending = await asyncio.wait(tasks, return_when=asyncio.FIRST_COMPLETED)
    for t in pending: t.cancel()        # cancel slower competitors
    return done.pop().result()
```

Section 5 — Agent Communication Protocol

```
from collections import defaultdict
from typing import Callable

class AgentBus:
    """
    In-process pub/sub message bus.
    For distributed deployment: replace with Redis pub/sub or Kafka.
    """
    def __init__(self):
        self._subscribers: dict[str, list[Callable]] = defaultdict(list)

    def subscribe(self, topic: str, handler: Callable):
        self._subscribers[topic].append(handler)

    async def publish(self, topic: str, message: AgentMessage):
        for handler in self._subscribers.get(topic, []):
            await handler(message)

    async def publish_result(self, result: AgentResult):
        """Convenience: publish result to role topic."""
        msg = AgentMessage(
            sender    = result.role,
            recipient = AgentRole.ORCHESTRATOR,
            msg_type  = "result" if result.success else "error",
            payload   = result.output,
            trace_id  = result.trace_id,
        )
        await self.publish(f"result.{result.role.value}", msg)

# MSG_TYPES (typed protocol between agents):
MSG_TYPES = {
    "task":     "Orchestrator -> Worker: new task",
    "result":   "Worker -> Orchestrator: success",
    "error":    "Worker -> Orchestrator: failure",
    "clarify":  "Worker -> User: needs human input",
    "progress": "Worker -> Orchestrator: streaming progress",
    "cancel":   "Orchestrator -> Worker: cancel in-flight task",
}

# TRACE_ID: links ALL messages for one user request across all agents
# trace_id=abc123
#   [t=0ms]    Orchestrator -> RetrievalAgent:  task
#   [t=0ms]    Orchestrator -> LTMemory:        fetch
#   [t=250ms]  RetrievalAgent -> Orchestrator:  result(schema_context)
#   [t=250ms]  LTMemory -> Orchestrator:        result(examples)
#   [t=250ms]  Orchestrator -> GenerationAgent: task(schema+examples)
#   [t=1050ms] GenerationAgent -> Orchestrator: result(sql)
#   [t=1050ms] Orchestrator -> ValidationAgent: task(sql+schema)
#   [t=1750ms] ValidationAgent -> Orchestrator: result(is_valid=True)
#   [t=1800ms] Orchestrator: execute + answer
#   Total: 1800ms vs ~3300ms sequential (45% faster)
```

Section 6 — Multi-Agent Evaluation

```
from dataclasses import dataclass

@dataclass
class MultiAgentMetrics:
    total_requests:     int   = 0
    success_rate:       float = 0.0   # fraction that produced a final answer
    phase1_latency_ms:  float = 0.0   # parallel retrieval + few-shot
    phase2_latency_ms:  float = 0.0   # generation
    phase3_latency_ms:  float = 0.0   # validation (all attempts)
    phase4_latency_ms:  float = 0.0   # execution
    total_latency_ms:   float = 0.0
    validation_retries: float = 0.0   # avg retries per request
    retrieval_quality:  float = 0.0   # recall@5 of RetrievalAgent

class MultiAgentEvaluator:
    async def evaluate(self, cases: list[dict]) -> MultiAgentMetrics:
        metrics = MultiAgentMetrics(total_requests=len(cases))
        n = len(cases)
        for case in cases:
            result = await self.orch.run(case["question"], case["schema_name"],
                                         "eval_tenant", "eval_user")
            if result["success"]:
                metrics.success_rate += 1/n
                for step in result["timeline"]:
                    phase = str(step["phase"]); lat = step.get("latency_ms",0)
                    if phase=="1":              metrics.phase1_latency_ms += lat/n
                    elif phase=="2":            metrics.phase2_latency_ms += lat/n
                    elif phase.startswith("3"): metrics.phase3_latency_ms += lat/n
                metrics.total_latency_ms += result["total_latency_ms"]/n
        return metrics

# PRODUCTION TARGETS (multi-agent system):
# success_rate          > 0.95    (95% get a final answer)
# total_latency_ms      < 4000    (faster than single-agent 8s via parallel P1)
# phase1_latency_ms     < 300     (retrieval + few-shot in parallel)
# phase2_latency_ms     < 1200    (generation with pre-filtered schema)
# phase3_latency_ms     < 800     (validation with smaller model)
# validation_retries    < 0.15    (< 15% of queries need retry)

# COST MODEL (key advantage of multi-agent specialization):
# Single-agent (all GPT-4o):   $X per query
# Multi-agent (gen=70B, others=8B):  ~0.35X per query (65% cost reduction)
# Only GenerationAgent needs the expensive model.
# RetrievalAgent filtering + ValidationAgent semantic check work fine on 8B.
```

Section 7 — Production Deployment

```
from asyncio import Semaphore

class AgentPool:
    """
    Pool of identical worker agents.
    Round-robin load balancing + concurrency limit via semaphore.
    In production: each pool on a separate GPU or Kubernetes pod.
    """
    def __init__(self, workers: list[BaseWorkerAgent], max_concurrent: int = 8):
        self.workers = workers
        self._sem    = Semaphore(max_concurrent)
        self._next   = 0

    async def run(self, task: TaskSpec) -> AgentResult:
        async with self._sem:
            worker     = self.workers[self._next % len(self.workers)]
            self._next += 1
            return await worker.run(task)

# Deploy agent pools independently:
retrieval_pool  = AgentPool([RetrievalAgent(...) for _ in range(4)],  max_concurrent=16)
generation_pool = AgentPool([GenerationAgent(...) for _ in range(2)], max_concurrent=4)
validation_pool = AgentPool([ValidationAgent(...) for _ in range(4)], max_concurrent=16)

# FastAPI production endpoint
from fastapi import FastAPI
app = FastAPI(title="Multi-Agent Text-to-SQL")

class QueryRequest(BaseModel):
    question: str; schema_name: str; tenant_id: str; user_id: str

@app.post("/query")
async def query(req: QueryRequest):
    result = await orchestrator.run(
        req.question, req.schema_name, req.tenant_id, req.user_id)
    if not result["success"]:
        raise HTTPException(status_code=500, detail=result["error"])
    return result

@app.get("/health")
async def health():
    return {"status":"ok","agents":{"retrieval":"healthy",
            "generation":"healthy","validation":"healthy"}}

# SCALING RULES:
# RetrievalAgent:  stateless, fast (250ms) — scale to many replicas, CPU-feasible
# GenerationAgent: GPU-bound, slow (1200ms) — scale carefully (expensive GPU)
# ValidationAgent: EXPLAIN is DB I/O bound — scale with DB connection pool
# Orchestrator:    stateless → horizontal scaling, no shared state
```

Section 8 — FAANG Multi-Agent Patterns

Google — Agent Space (Vertex AI): Independent Scaling per Agent Type

In Google's internal Search Ads quality pipeline, specialized agents handle query understanding (BERT-based), ad relevance scoring (custom embedding), and policy compliance checking (rule-based + LLM) in parallel. The orchestrator is stateless — it routes tasks and merges results only. Compliance checking runs on CPU (cheap); relevance scoring autoscales 10-400 GPU replicas. Each agent type scales independently — the orchestrator sees only the merged result.

Microsoft — AutoGen Framework: Conversational Multi-Agent Protocol

AutoGen agents communicate via natural language messages (not structured JSON). In Microsoft Copilot for Finance, three agents collaborate: Data Retrieval (fetches from Dynamics 365 ERP), Analysis (trends + calculations), Report (formats as Word/Excel). Agents can ask each other clarifying questions mid-task — making the conversation graph non-linear. Result: 23% accuracy improvement on complex financial queries vs a single monolithic prompt.

Amazon — Bedrock Sub-Agents: Supervisor Routing to Specialists

Bedrock's supervisor agent invokes specialized sub-agents as tools. In Amazon's supply chain system, supervisor routes to: Demand Forecasting Agent (historical sales), Inventory Agent (warehouse systems), External Factors Agent (weather, events, promotions) — each with its own RAG knowledge base and tools. The supervisor merges outputs into a final forecast. Result: 67% reduction in supervisor context window usage vs stuffing everything into one agent.

Meta — Multi-Agent Debate: Two Agents Review Each Other

Two LLM instances receive the same question independently. Each produces an answer. They then see each other's answers and produce revised answers (2-3 rounds). A synthesis agent produces the consensus final answer. On GSM8K math: 2-agent debate improved accuracy from 77.6% to 88.9% — equal to a model 5x larger. Mechanism: one agent's error is often caught by another agent reasoning from a different starting point. Used in Meta's internal fact-checking pipeline for news content.

Netflix — Parallel A/B Test Analysis: Three Agents, One Verdict

When an A/B test concludes, three agents run simultaneously: Statistical Agent (Bayesian inference, p-values, confidence intervals), Business Metrics Agent (revenue impact, subscriber retention, engagement), Risk Agent (novelty effects, population contamination, SRM checks). A Synthesis Agent merges all three into a recommendation. The Risk Agent can veto even if Statistical + Business both agree. Result: analysis time reduced from 4 hours to 45 minutes. Replaced a sequential pipeline that blocked on each step.

Section 9 — Architect's Lens

```
WHEN TO USE MULTI-AGENT vs SINGLE AGENT:
  Single agent: < 5 tool calls, sequential, one domain, latency target < 3s
  Multi-agent:  > 5 tool calls, parallel phases needed, latency target > 5s

WORKER AGENT MODEL SIZING:
  RetrievalAgent:   filter + classify task → small model (2B-7B), CPU-feasible
  GenerationAgent:  SQL generation, CoT → best model you can afford (70B or GPT-4o)
  ValidationAgent:  pattern match + EXPLAIN → medium model (8B), mostly rule-based
  Cost saving: only GenerationAgent uses expensive model → 60-70% cost reduction

FAILURE ISOLATION (critical in multi-agent):
  RetrievalAgent fails   → fallback: broad retrieval (top_k=20), retry once
  GenerationAgent fails  → return error cleanly; NEVER return hallucinated SQL
  ValidationAgent fails  → skip validation with warning; return generation output
  Rule: one agent's failure must NEVER silently corrupt the final output

CONTEXT PASSING BETWEEN AGENTS:
  GOOD: pass only what the next agent needs (schema_context string, not raw tables list)
  BAD:  pass entire previous agent's output (bloats context unnecessarily)
  Rule: each agent's input = minimum context needed to do its job

STATELESS ORCHESTRATOR (required for production scale):
  Stateful orchestrator → single point of failure; can't scale horizontally
  Stateless orchestrator: all context in the message payload + Redis (trace_id → state)
  Benefits: restart a failed run from any phase; horizontal scaling; replay

LATENCY MODEL (parallel Phase 1):
  Phase 1 sequential: retrieval(250ms) + few-shot(200ms) = 450ms
  Phase 1 parallel:   max(retrieval, few-shot) = 250ms      (44% faster)
  RULE: gather() only saves time when parallel tasks have UNEQUAL durations.
  If retrieval=250ms and few-shot=250ms: parallel=250ms, sequential=500ms (50% gain)
  If both =250ms but one blocks the other: no gain — check for DB connection sharing

COMMUNICATION PROTOCOL CHOICE:
  Same process:     asyncio.gather() + in-process function calls (lowest latency)
  Same machine:     AgentBus (in-process pub/sub)
  Different pods:   Redis pub/sub or Kafka (adds ~2ms latency per hop)
  Different regions:message queue (SQS / Pub/Sub) — accept higher latency for reliability
```

Section 10 — Exercises and Summary

Exercise 1: Build the Orchestrator Timeline

```
result = await orchestrator.run("Monthly active users by country","analytics",...)
for step in result["timeline"]:
    print(f"Phase {step['phase']} | {step['agent']:20s} | {step['latency_ms']:.0f}ms")
# Expected:
# Phase 1   | retrieval+few_shot  | 243ms
# Phase 2   | generation          | 1187ms
# Phase 3.1 | validation          | 712ms
# Phase 4   | execute             | 58ms
# Total: 2200ms  (vs ~3300ms single agent)
```

Exercise 2: Inject a Retrieval Failure

```
# Monkeypatch RetrievalAgent to return success=False
# Verify: result["success"]==False and result["error"]=="Retrieval failed"
# Then implement fallback: broad retrieval (top_k=20), retry once
# Verify fallback recovers the query successfully
```

Exercise 3: Small vs Large Model per Agent

```
# Run same 20 questions with 3 configurations:
# A: all agents = LLaMA-3-70B    (expensive, high accuracy)
# B: generation=70B, others=8B   (recommended)
# C: all agents = LLaMA-3-8B     (cheap)
# Measure: accuracy (correct_kw), cost (tokens*price), latency
# Expected: Config B = ~95% of Config A accuracy at 35% of Config A cost
```

Summary

Concept	Key Point
Multi-agent vs single	Single: sequential, <5 tools. Multi: parallel, specialization, fault isolation
AgentRole enum	ORCHESTRATOR / RETRIEVAL / GENERATION / VALIDATION / CRITIC
AgentMessage	Typed message: sender, recipient, msg_type, payload, trace_id
TaskSpec	Orchestrator→worker: question + schema + context from prior agents
AgentResult	Worker→orchestrator: success, output, latency_ms, error
RetrievalAgent	Sole job: retrieve + filter schema to exactly needed tables/columns
GenerationAgent	Sole job: SQL generation with CoT from pre-filtered schema
ValidationAgent	Sole job: EXPLAIN + LLM semantic check; corrected_sql on failure
TextToSQLOrchestrator	Phase1(parallel) → Phase2(gen) → Phase3(val) → Phase4(exec)
asyncio.gather()	Parallel Phase 1: retrieval + few-shot simultaneously (44% faster)
Speculative execution	N agents compete; first valid result wins; rest cancelled

AgentBus	In-process pub/sub; replace with Redis/Kafka for distributed
AgentPool	Round-robin + semaphore; independent scaling per agent type
SLOs	success>95%, latency<4s, phase1<300ms, retries<15%
Cost model	Only GenerationAgent needs 70B; others 8B → 60-70% cost reduction
Stateless orchestrator	State in message+Redis → horizontal scaling + replay

NEXT: Day 21 — Week 3 Capstone

Build and evaluate the complete end-to-end Text-to-SQL system (Days 13-20 integrated): real SPIDER evaluation, execution accuracy, latency SLOs, cost per query, and production readiness checklist.